

Smart Chef HR — Гайд по работе с ботом

- Smart Chef HR — Гайд по работе с ботом
 - 0. TL;DR — что куда жать
 - 1. Как кандидат и работодатель «переписываются» 
 - 2. Как админ всё мониторит 
 - 3. Где давать одобрение на компанию 
 - 4. Как получить права на бота 
 - 5. Если нужно вносить изменения 
 - 6. Может ли админ вручную создать вакансию? 
 - 7. Сколько бот «помнит» — память бота 
 - 8. Сценарий «по полочкам»
 - 9. Частые «а что если» 
 - 10. Что недавно изменилось 

Smart Chef HR — Гайд по работе с ботом

Короткий, практичный гайд: **как боту жить со всех сторон** — кандидат, работодатель, админ, владелец. Бот: [@hr_bibot](#) · команда [/admin](#) открывает админ-панель.

0. TL;DR — что куда жать

Кто я	Что делаю	Что нажимаю
Новый пользователь	Открываю бота	<code>/start</code> → выбираю роль
Соискатель	Заполнил анкету и хочу смотреть вакансии	<code>/search</code> или «🔍 Смотреть вакансии»
Соискатель	Посмотреть/изменить анкету	<code>/profile</code>
Работодатель	Создать новую вакансию	«+ Новая вакансия»
Работодатель	Посмотреть свои вакансии	«📅 Мои вакансии»
Обе стороны	Посмотреть взаимные отклики	«💚 Мои отклики»
Админ	Открыть админ-панель	<code>/admin</code>
Админ	Узнать свой Telegram ID	<code>/myid</code>

1. Как кандидат и работодатель «переписываются»

Никак напрямую — это принципиально. Бот = посредник, никто не видит чужой Telegram-ник или телефон до того, как:

1. Обе стороны **поставили** «💚 **Нравится**» на карточке друг друга (взаимный матч).
2. У смотрящего активна **подписка** (1 000 ₮ кандидат / 10 000–20 000 ₮ работодатель). Без оплаты адрес и телефон скрыты звёздочками.
3. Работодатель **назначил собеседование** через кнопку «📅 Предложить время» под карточкой матча.
4. Кандидат выбрал один из 1–3 слотов времени.

После этого: - Бот шлёт **обеим сторонам** карточку: «Собеседование в такой-то день, по адресу X, у компании Y». - Бот сам шлёт **напоминания** за 24 ч и за 2 ч до встречи.

Чата между сторонами в боте нет. Контакт (телефон) появляется в карточке после оплаты — дальше они звонят/пишут вне бота.

Логика: [app/services/matching.py](#), [app/services/scheduler.py](#), [app/handlers/interview.py](#).

2. Как админ всё мониторит

Открываете `/admin` — появляется **меню с 8 кнопками** (inline). Жмёте — получаете список.

Что под кнопками

Кнопка / команда	Что показывает
 На модерации / <code>/pending</code>	Новые компании, ждущие одобрения. Под каждой —  <code>Одобрить</code> и  <code>Отклонить</code> .
 Активные вакансии / <code>/vacancies_active</code>	Все вакансии в статусе ACTIVE и IN_PROGRESS (последние 30).
 Закрытые вакансии / <code>/vacancies_closed</code>	Архив закрытых вакансий.
 Соискатели / <code>/candidates_active</code>	Активные анкеты кандидатов: имя, город, должность, контакт.
 Отклики и матчи / <code>/applications</code>	Кто кого лайкнул, score матча, реакция работодателя, статус собеса.
 Подписки и оплаты / <code>/subscribers</code>	Кто платит, до какой даты, сколько контактов открыл.
 Заявки на вывод / <code>/payouts</code>	Кто из рефералов просит вывод на Kaspi. <code>/paid <id></code> — закрыть.
 Подсказка / любая команда	Полный список команд.

Ручные команды (для точечных операций)

```
/history <tg_id> [limit]    – последние N сообщений бот+пользователь
/grant <tg_id> <tariff>     – выдать подписку вручную
    tariff: candidate | employer_basic | employer_extended
/logic                      – куда смотреть, чтобы менять бизнес-логику
```

Где живёт зеркало (если включён Google Sheets)

4 листа: **Applicants** / **Employers** / **Vacancies** / **Interviews**. ID листа задаётся в `.env` (`GOOGLE_SHEET_ID`), доступ — через service-account JSON. Это копия БД для удобного просмотра/фильтрации в браузере.

Логика: [app/handlers/admin.py](#), [app/services/google_sheets.py](#).

3. Где давать одобрение на компанию

Каждый раз, когда **новый работодатель** заканчивает регистрацию + создаёт первую вакансию:

1. Его вакансия уходит в статус **IN_PROGRESS** (**не публикуется**, не показывается кандидатам).
2. **Всем админам** из **ADMIN_IDS** падает уведомление с карточкой компании (БИН, город, контакт) и кнопками **✓ Одобрить** / **✗ Отклонить**.
3. Нажали **«Одобрить»**:
 - статус компании → APPROVED
 - все её «висящие» вакансии → ACTIVE
 - бот публикует вакансии и сразу прогоняет матчинг по кандидатам
 - работодателю приходит сообщение «**✓** Компания одобрена»
4. Нажали **«Отклонить»**:
 - бот спрашивает у вас причину текстом
 - работодатель получает «**✗** Не прошли проверку. Причина: ...»

Если уведомление потерялось — **/admin** → ⌚ **На модерации** покажет всех висящих.

Логика: [app/services/verification.py](#), [app/handlers/admin.py:cmd_pending](#).

4. Как получить права на бота 🗝

1. **Узнать свой Telegram ID.** Откройте бота и напишите **/myid**. Бот ответит числом — это ваш **tg_id**.
2. **Добавить ID в .env.** Откройте файл **.env** (или **.env** в Railway / Docker secrets, если деплой удалённый) и добавьте ID в список:

```
ADMIN_IDS=[647787396,866175753]
```

Формат — JSON-массив целых чисел через запятую. Уже добавлены: - **647787396** — основной владелец - **866175753** — новый админ (только что добавлен)

3. **Перезапустить бота.** Конфиг читается при старте процесса: - Локально: **make run** (или Ctrl+C → снова **python -m app**) - Docker: **docker compose restart bot** - Railway: «Redeploy» в дашборде

4. **Проверить.** В Telegram напишите боту **/admin** — должно появиться меню. Если «команда не распознана» — ID не подхватился, перепроверьте **.env**.

Логика проверки: [app/handlers/admin.py:31-33](#) —
`router.message.filter(F.from_user.id.in_(ADMIN_IDS))` .

5. Если нужно вносить изменения

Файл [CLAUDE.md](#) — карта проекта. Команда `/logic` в боте показывает то же самое короче.
Главное:

Что менять	Куда смотреть
Тексты сообщений пользователю	app/locales/ru.py — одно место для всех строк UI
Цены, лимиты, бонусы	<code>.env (TARIFF_* , EMPLOYER*_LIMIT , REFERRAL_BONUS)</code>
Алгоритм подбора (score, веса)	app/services/matching.py
Доступ и блокировки	app/services/access_control.py
Вопросы анкеты соискателя	app/states/candidate.py + app/handlers/candidate.py
Регистрация работодателя и вакансии	app/states/employer.py + app/handlers/employer.py
Платежи / провайдер	app/services/payments.py , <code>.env:</code> <code>PAYMENT_PROVIDER</code>
Кнопки и клавиатуры	app/keyboards/inline.py , app/keyboards/reply.py
Админ-команды	app/handlers/admin.py

Поменять цену подписки за минуту

В `.env` :

```
TARIFF_CANDIDATE=1500
TARIFF_EMPLOYER_BASIC=12000
```

Перезапустить бот. Готово — старые подписки не меняются, новые покупки идут по новой цене.

Добавить новую должность в HoReCa

В `app/core/constants.py` найти `HORECA_POSITIONS` и добавить пару `"sushist": "🍣 Сушист"`.
Перезапустить.

Поменять текст приветствия

`app/locales/ru.py` → ключ `start_greeting`. Один раз — везде.

6. Может ли админ вручную создать вакансию? 🤔

Нет, и это сознательно. Архитектура: вакансия привязана к работодателю, работодатель привязан к Telegram-пользователю. «Левых» вакансий без владельца в системе быть не должно — иначе: - некому отвечать на отклики кандидатов; - некому платить за подписку и лимиты; - невозможно назначить собеседование (кому идти?).

Что админ может вместо этого:

1. **Создать тестовую вакансию** — стать работодателем с другого аккаунта, пройти регистрацию, сам себя одобрить.
2. **Активировать подписку без оплаты** — `/grant <tg_id> employer_basic` выдаст подписку на 30 дней. Полезно для тестов и «подарков» партнёрам.
3. **Закрыть/переоткрыть чужую вакансию** — пока не реализовано через бота. При острой необходимости — в БД:

```
UPDATE vacancies SET status = 'closed' WHERE id = 42;
```

Если нужна возможность «админ создаёт вакансию за работодателя» — это отдельная фича, скажите, добавлю команду `/create_vacancy <employer_id>` с FSM-формой.

7. Сколько бот «помнит» — память бота 🧠

У бота **три слоя памяти**, у каждой свой срок:

7.1 FSM — состояние диалога (Redis)

- **Что хранит:** где сейчас пользователь в анкете («ввёл имя, ждём город»), временные ответы.
- **Срок:** дефолт `RedisStorage` — **бесконечно, пока не очистится явно**. `/cancel` или `/start` чистят. Заполнение анкеты переживает рестарт бота.

- **Где:** Redis, ключи вида `fsm:user:<tg_id>:state`.

7.2 История диалога (PostgreSQL)

- **Что хранит:** каждое сообщение бот↔пользователь — текст, тип (text/photo/callback), время.
- **Срок: навсегда** (TTL не настроен). Таблица `conversation_messages`, индекс по `(user_id, created_at)`.
- **Зачем:** контекст для AI-поддержки (последние 8 сообщений отдаются в Gemini), `/history` для админа.
- **Просмотр:** `/history <tg_id> 50` — последние 50 сообщений выбранного пользователя.
- **Очистка** (если когда-то понадобится):

```
DELETE FROM conversation_messages WHERE created_at < NOW() - INTERVAL '90 days';
```

7.3 Бизнес-сущности (PostgreSQL)

- Анкеты, вакансии, матчи, собеседования, подписки, платежи, рефералы.
- **Срок:** навсегда. Это «база» — никогда не удаляется автоматически.
- **Закрытые вакансии и старые матчи** просто меняют `status`, в БД остаются.

7.4 Что НЕ помнит

- **AI-ответы Gemini** не персистируются как «знание» — каждый запрос идёт заново с текущим контекстом из БД.
- **Сессии без авторизации** — нет, любая активность создаёт `User` при первом сообщении.
- **Пароли/токены пользователей** — их просто нет: Telegram авторизует сам.

7.5 Объём

- 1 пользователь активного дня $\approx$ 20–50 сообщений в `conversation_messages` $\approx$ 5–10 KB.
- 1 000 активных пользователей в месяц $\approx$ 300 MB истории. Для PostgreSQL — копейки.

Логика: [app/services/conversation.py](#), [app/db/models/conversation.py](#).

8. Сценарий «по полочкам»

Кандидат с нуля

1. `/start` → «👤 Ищу работу»
2. Заполняет анкету: имя → возраст → город → район → должность → опыт → график → ожидаемая ЗП → «о себе» → фото → телефон.
3. Карточка сохранена. Кнопка «🔍 Смотреть вакансии».
4. Листает: ❤️ / ⏮️ / ⏭️ / ❌.
5. При совпадении — приходит уведомление «❤️ Взаимный интерес!». Без подписки телефон и адрес скрыты.
6. Покупает подписку (1 000 ₼) → видит контакты и адреса.
7. Работодатель предлагает 1–3 слота времени → кандидат выбирает.
8. Бот шлёт напоминания за 24 ч и 2 ч.

Работодатель с нуля

1. `/start` → «👤 Ищу сотрудника»
2. Регистрация: компания → контактное лицо → телефон (через «Поделиться контактом») → город.
3. Создаёт вакансию: должность → требования → условия → ЗП от → ЗП до → график → адрес → фото.
4. Подтверждает. Вакансия уходит в `IN_PROGRESS`, админ получает заявку.
5. Админ одобряет → вакансия `ACTIVE`, прогон матчинга, уведомления подходящим кандидатам.
6. Появляются отклики → «❤️ Мои отклики» → «📅 Предложить время».
7. Покупает подписку (`employer_basic` 10 000 ₼ / 20 откликов или `employer_extended` 20 000 ₼ / 50).
8. После каждого открытия контакта — счётчик `candidates_viewed += 1`. На пределе — апсейл на extended.

Админ повседневно

1. `/admin` — открывает меню.
 2. 🕒 **На модерации** — одобряет/отклоняет.
 3. ❤️ **Отклики и матчи** — смотрит, где зависло (матч есть, собес не назначен — пишет работодателю).
 4. 💰 **Подписки и оплаты** — кто платит, когда продлится.
 5. 📄 **Заявки на вывод** — обрабатывает Kaspi, отмечает `/paid <id>`.
 6. По жалобе пользователя — `/history <tg_id> 30` смотрит, что бот ему писал.
-

9. Частые «а что если»

Ситуация	Решение
Работодатель не получил «Одобрено»	<code>/pending</code> → нажать кнопку повторно. Если уже одобрен — <code>/vacancies_active</code> , должна быть в списке.
Кандидат жалуется «нет вакансий»	Возможно, нет совпадений по городу+должности. Проверить через <code>/applications</code> , что для него в принципе есть матчи.
Случайно одобрили не ту компанию	Прямо в БД: <code>UPDATE employers SET verification_status='rejected' WHERE id=X; + UPDATE vacancies SET status='in_progress' WHERE employer_id=X;</code>
Бот не отвечает на <code>/admin</code>	ID не в <code>ADMIN_IDS</code> или бот не перезапущен после правки <code>.env</code> .
Платёж пришёл, подписка не активировалась	<code>/grant <tg_id> <tariff></code> — выдать вручную.
Нужно почистить старые сообщения	<code>DELETE FROM conversation_messages WHERE created_at < NOW() - INTERVAL '180 days';</code>

10. Что недавно изменилось

- ✓ Админ `866175753` добавлен в `ADMIN_IDS` (нужно **перезапустить бота**, чтобы применилось).
- ✓ Команда `/admin` теперь показывает **inline-меню с 8 кнопками** вместо стены текста. Все команды остались работать как раньше.
- ✓ Добавлен этот гайд — `GUIDE.md`.

После перезапуска нового админа `866175753` ждёт `/admin` с готовой панелью.